HW3

Due November 6 at 11:55 PM

Procedural 3D Robotic Arm Kinematics & Lighting

In this assignment, you will build an interactive 3D robotic arm assembly in C++ using OpenGL for rendering and SFML for window creation and user interaction. All component geometry must be procedurally generated, organized hierarchically, and rendered using Phong shading with texture mapping.

You are to work on this assignment individually. To make the grading process easier, please read the submission instructions carefully. To ensure adherence to academic integrity guidelines, all submissions must include an accompanying declaration and reflection statement.

---

1. Kinematic Tree Structure & Hierarchy

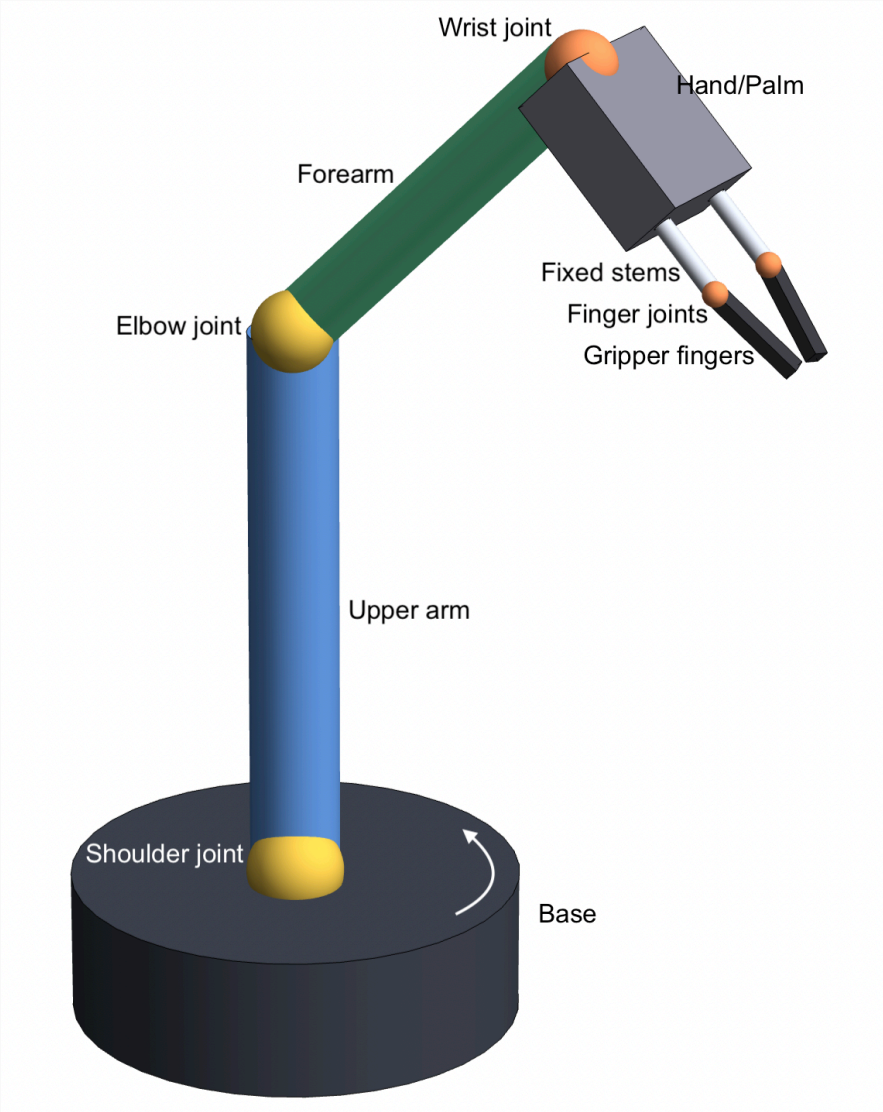
The robotic arm is structured as a hierarchical kinematic tree. Each child node accumulates the transformation matrices of its ancestors down the chain:

```
World Origin
└── Base Cylinder (Root Joint: Rolling / Rotation around the world's vertical)
    └── Shoulder Joint (Sphere) & Upper Arm Cylinder (Shoulder Pitch Joint)
        └── Elbow Joint (Sphere) & Forearm Cylinder (Elbow Pitch Joint)
            └── Wrist Joint (Sphere) & Hand Assembly (Wrist Pitch Joint)
                ├── Palm (Cuboid)
                ├── Fixed Stems (Cylinders) & Finger Joints (Spheres)
                └── Upper & Lower Fingers (Cuboids) (Gripper Open/Close Joint)
```

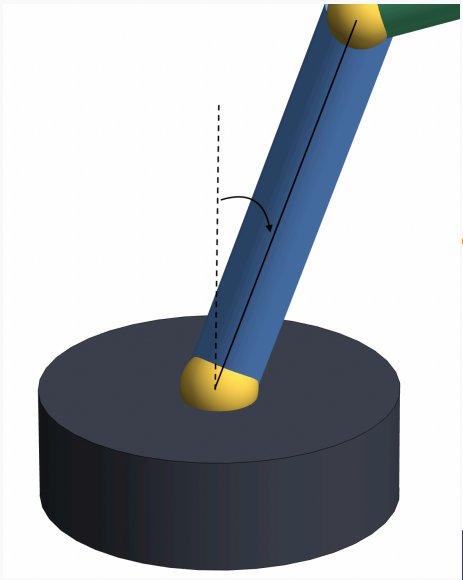
Note: Some joint angles may result in non-physical configurations or self-intersections. This is acceptable for this assignment.

1.1 Visual Reference & Screenshots

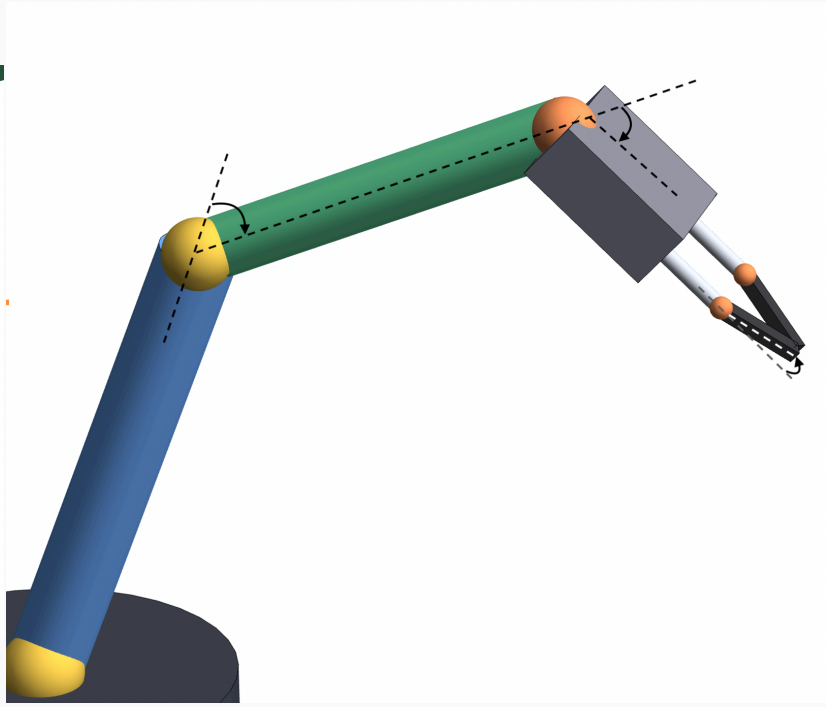
Below are reference renders of the robotic arm model created in Mathematica to demonstrate the expected component layout and articulated poses:



Full view of the arm showing the various components and how they connect. The base rests flat and can rotate about the global vertical axis.



The upper arm pitches relative to the global vertical axis.



The forearm pitches relative to the principal axis of the upper arm, the hand/palm pitches relative to the principal axis of the forearm, and the grabber fingers rotate symmetrically about the finger joints.

---

2. Implementation Specifications

2.1 Starter Code

The sfml-starter-with-opengl-3d repository provides much of the setup and organization needed to manage geometry and shaders for 3D rendering. Please use this repo as starter code and extend it to implement the robot arm.

2.1 Keyboard Controls & Motion Limits

Implement keyboard controls using SFML to manipulate the individual joints. You are responsible for determining and implementing appropriate minimum and maximum angular ranges for each control so that a full, expressive range of motion is achievable. You do not need to worry about dependence between the various angles to prevent self-intersections, but do restrict angular ranges where such ranges make sense, e.g. the grabber fingers have a limited range of motion and the upper arm cannot pitch below the base.

Joint / Component	Motion Description	Controls	Angular Limits Requirement
Base Roll	Rolls / rotates the entire assembly around the global vertical	`A` / `D` or `Left` / `Right`	Determine range for full rotation
Shoulder Pitch	Pitches upper arm up/down	`W` / `S`	Determine sensible range for pitching
Elbow Pitch	Pitches forearm up/down relative to upper arm	`R` / `F`	Determine sensible range for pitching
Wrist Pitch	Pitches entire hand/palm assembly relative to the upper arm.	`T` / `G`	Determine sensible range for pitching
Gripper Claw	Symmetrically opens and closes fingers	`O` (Open) / `C` (Close)	Determine open/close angle limits
Reset Pose	Resets all joint angles to default (0°)	`Space`	N/A

2.2 Procedural Geometry, Normals & Texture Coordinates

All geometry must be procedurally generated at runtime. Do not load external 3D object files (.obj, .fbx, .gltf, etc.). Every vertex in your vertex attributes must include 3D position (x, y, z), surface normal (N_x, N_y, N_z), and 2D texture coordinates (u, v).

Normal Generation & Shading Guidance:

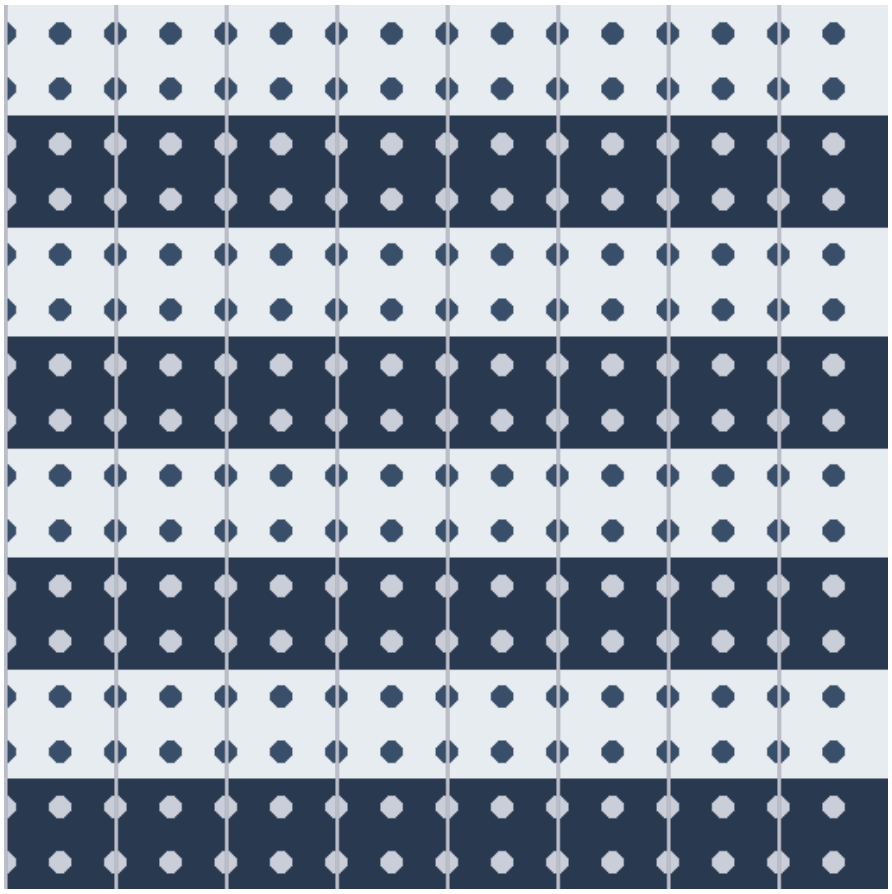
- **Flat Surfaces (Cuboids, Cylinder Caps):** For flat planar faces, duplicate vertices along sharp edges so that vertices are not shared across adjacent faces. All 3 vertices belonging to a single flat triangular face should be assigned the exact same face normal orthogonal to that face (e.g., $N = (0, 0, 1)$ for top caps, or $N = (1, 0, 0)$ for a cuboid's side face).
- **Curved Surfaces (Cylinder Side Walls, Spheres):**
  - *Cylinder Side Wall:* Vertices along the curved wall should share radial unit normals pointing directly outward from the central cylinder axis: $N = (\cos\theta, \sin\theta, 0)$.

  - *Sphere:* Vertices on a sphere should share radial unit normals pointing outward from the sphere center C: $N = \frac{P-C}{\|P-C\|}$.
- **Unified Shader Pipeline:** Passing per-vertex normals into your GLSL Phong shader will naturally render flat surfaces with sharp planar lighting and curved surfaces with smooth specular highlights—allowing a single fragment shader to render all components seamlessly. For untextured components, bound texture samplers can be bypassed.

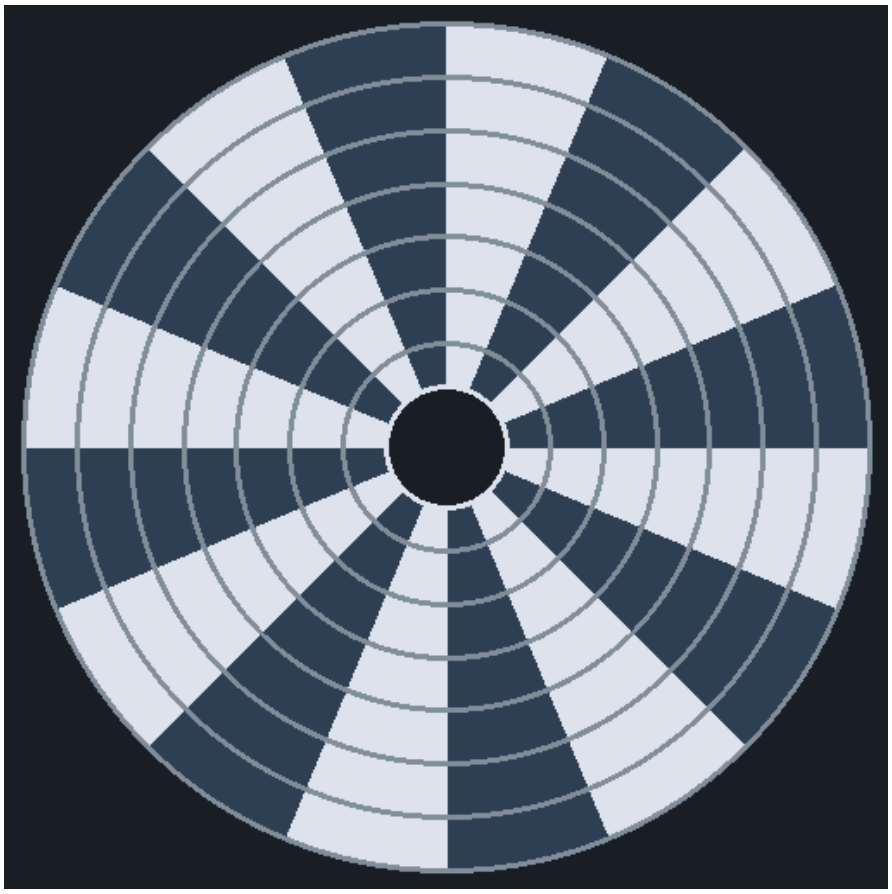
2.3 Texture Images

You are provided with two 512×512 PNG texture files.

1. `texture_arm_linkage.png`: Apply this to the cylindrical arm linkages (Upper Arm and Forearm) to make cylinder rotation clearly visible. Scale the texture coordinates appropriately to
   make the texture pattern apparent.



2. `texture_base_top.png`: Apply this to the top circular face of the Base cylinder to make base roll rotations easily visible.



2.4 Lighting & Shading (Phong Model)

Implement Phong shading in GLSL (per-fragment lighting) with at least one light source in the scene:

- **Textured Components:** For textured elements (Base top and Arm cylinders), scale or replace the *diffuse color component* in the Phong calculation using the color sampled from the texture sampler.
- **Untextured Components:** Assign appropriate material properties (ambient, diffuse, specular coefficients, and shininess exponent) to non-textured elements (Joint spheres, Palm, Stems, Fingers) as desired.

2.5 Camera Setup

Position and orient the camera so that the entire robotic arm assembly remains clearly visible inside the view frustum throughout all manipulations without clipping.

2.6 Reference Dimensions

You may use the following recommended relative proportions for your component geometry (or adjust them slightly as long as proportions remain consistent):

Component	Dimension Property	Sample Value
Base	Height / Radius	0.70 / 1.10
Shoulder Joint	Sphere Radius	0.24
Upper Arm	Length / Radius	2.80 / 0.22
Elbow Joint	Sphere Radius	0.20
Forearm	Length / Radius	2.25 / 0.17
Wrist Joint	Sphere Radius	0.17
Palm	Length / Width / Height	0.82 / 0.36 / 0.68
Fixed Stems	Length / Radius	0.58 / 0.045
Finger Joints	Sphere Radius	0.06
Fingers	Length / Width / Thickness	0.80 / 0.085 / 0.07

---

3. Submission, Demo and Grading

- **Submission:**
  Please submit via Gradescope. Please take note of the following when preparing your submission.
  - Please submit your code and declaration as a self-contained package/repository that can be built/compiled on its own via CMake just like the starter code.
  - You do not need to include any helper libraries (GLM, SFML, stb_image) that are fetched by the CMake build process.
  - Do include all shaders and textures that are needed to run your program.
  - Do include your declaration file.
  - Optionally include a README file in case there are any other interaction controls or caveats you would like to point out.

- **Demo: (5 points + 5 bonus)**
  You will be required to demo your program to your TA during a scheduled tutorial session in MS 239. Please be prepared to walk your TA through your implementation during the demo.
  You can earn bonus points by demoing early.

- **Grading:**
  The following high-level grading scheme will be used to grade the functionality of your implementation.
  - Camera (3 points)
  - Procedural geometry generation (5 points)
  - Model transformations + interactions (7 points)
  - Texturing + lighting and shading (5 points)
  - Declaration and reflection (2 points)